

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	A.VEERASANTHOSH	Reg. No.:	192525187
Section / Batch:		Date:	20/07/2026

Code of Conduct

I A.VEERASANTHOSH / 192525187 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

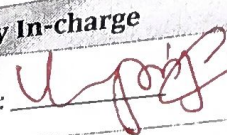
Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

Annexure A – Analytical Problem Solving (Own Entry)

S.No	REG NO	NAME	ANALYTICAL PROBLEM SOLVING
39	192525187	VEERA SANTHOSH A	1. Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. 2. A queue implemented using an array report "Queue Full" even though some positions are empty. ➤ Analyze why this occurs. ➤ Explain how a circular queue solves this problem.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Linear Queue vs circular Queue

Linear Queue Behavior

In a linear queue front only moves forward on dequeue and never wraps around only moves forward on enqueue

operation	front	rear	Array Utilization
Enqueue 1	0	0	index 0 used
Dequeue 1	1	0	index 0 now dead space
Enqueue 2	1	1	index 1 used
Dequeue 2	2	1	index 1 now dead space

- Every alternating enqueue - dequeue pair advances both front and rear by one position, but the freed slots never reused
- After just n such operations, rear reaches the end of the array (index $n-1$) and any empty

• For 100 alternating operations you'd need an array of size ≥ 50 (since rear advances once per enqueue) just to survive 50 enqueue - the array size limits total lifetime enqueues, not concurrent elements.

• Time complexity per operation is still $O(1)$, but space is wasted catastrophically. effective utilization trends toward 0% as operations continue

Circular Queue behaviour

$$\text{rear} = (\text{rear} + 1) \% n$$

$$\text{front} = (\text{front} + 1) \% n$$

• After a dequeue frees a slot - is immediately reusable once rear wraps back around to it.

• For the same alternating patterns an array of size as small as 2 (or a small constant) can sustain all 100 operations indefinitely recycled

Why a Linear Queue Falsely Reports "Queue Full"

Root cause analysis

In linear (non-circular) array-based queue, the full condition is typically checked as:

if ($\text{rear} == n-1$) $\rightarrow$ Queue Full

the problem:

front never resets or moves backward to reclaim space. Every dequeue increments front, permanently abandoning that index

rear only increases, moving toward the end of the array with every enqueue ... regardless of how much space has been freed at the beginning.

once rear hits index $n-1$, the check $\text{rear} == n-1$ triggers "Queue Full" even if front is now at index 40 out of a 50 element array, meaning 40 slots are

are sitting empty and unused

A circular queue treats the array as circular / wrap-around, connecting index $n-1$ back to index 0 conceptually

enqueue: $rear = (rear + 1) \% n$

dequeue: $front = (front + 1) \% n$

1. Redefined Space is reused: When rear reaches the end of the array,

instead of failing it wraps to index and if that slot was freed by an earlier dequeue it's immediately available.

2. Full / empty condition are redefined using element count or a size tracker (not raw pointer position)

Empty: $front == rear$ and $count == 0$

Full: $count == n$ or equivalently

$(rear + 1) \% n == front$